ANEXO PAM

📙Descargar PDF

- ANEXO PAM
  - INTRODUCCIÓN
  - ARQUITECTURA GENERAL DE PAM
  - ORIGEN DE LA CONFIGURACIÓN
    - `/etc/pam.conf`
    - `/etc/pam.d/`
  - ESTRUCTURA DE LAS REGLAS PAM
    - FORMATO EN `/etc/pam.conf`
    - FORMATO EN `/etc/pam.d/NOMBRE_SERVICIO`
  - SIGNIFICADO DE LOS CAMPOS
    - CAMPO `SERVICE`
    - CAMPO `TYPE`
    - CAMPO `CONTROL`
      - SINTAXIS SIMPLE
      - SINTAXIS EXTENDIDA
    - CAMPO `MODULE-PATH`
    - CAMPO `MODULE-ARGUMENTS`
  - CÓMO SE EVALÚA UNA PILA PAM
    - IDEA GENERAL DE PILA O STACK
    - EVALUACIÓN DE `REQUIRED`
    - EVALUACIÓN DE `REQUISITE`
    - EVALUACIÓN DE `SUFFICIENT`
    - EVALUACIÓN DE `OPTIONAL`
  - DIFERENCIA ENTRE `INCLUDE` Y `SUBSTACK`
  - ORDEN DE EVALUACIÓN PRÁCTICO
  - ERRORES DE SINTAXIS Y PRECAUCIONES
  - QUÉ NO HACE ESTE ANEXO
  - REFERENCIAS

INTRODUCCIÓN

PAM son las siglas de **Pluggable Authentication Modules**. Según la documentación oficial de Linux-PAM, PAM proporciona una forma de desacoplar las aplicaciones que conceden privilegios de los mecanismos concretos de autenticación, autorización y gestión de sesión que necesitan.

La idea central es sencilla: una aplicación compatible con PAM no implementa por sí misma toda la lógica de autenticación. En su lugar, delega esa lógica en la biblioteca PAM, y esta consulta una configuración que indica qué módulos deben ejecutarse y cómo debe evaluarse el resultado de cada uno.

Esta arquitectura permite cambiar políticas de acceso sin reescribir la aplicación. El mismo servicio puede apoyarse en contraseñas locales, restricciones horarias, comprobaciones de cuenta, acciones al abrir sesión o integración con sistemas externos, todo ello mediante configuración y módulos.

ARQUITECTURA GENERAL DE PAM

Desde el punto de vista funcional, PAM se apoya en tres piezas:

- La **aplicación** o servicio compatible con PAM, por ejemplo `login`, `su`, `sshd` o `vsftpd`.
- La **biblioteca PAM**, que recibe las llamadas de la aplicación y coordina el proceso.
- Los **módulos PAM**, que son bibliotecas compartidas cargadas según la configuración definida para cada servicio.

Cuando una aplicación PAM-aware se inicia, se enlaza con la API de PAM y la biblioteca lee la configuración correspondiente. La propia documentación de `pam.conf(5)` y `pam.d(5)` indica que esa configuración puede proceder del fichero `/etc/pam.conf` o, de forma alternativa, del directorio `/etc/pam.d/`. Si existe `/etc/pam.d/`, Linux-PAM ignora `/etc/pam.conf`.

Por tanto, en la práctica habitual cada servicio suele tener su propio fichero dentro de `/etc/pam.d/`, y ese fichero define la pila de módulos que PAM deberá evaluar para ese servicio.

ORIGEN DE LA CONFIGURACIÓN

`/etc/pam.conf`

Es el formato histórico centralizado. En él pueden definirse reglas para varios servicios dentro de un único fichero.

`/etc/pam.d/`

Es el formato más flexible y el más habitual. Cada fichero del directorio representa la configuración de un servicio concreto. Según la documentación oficial, el nombre del fichero actúa como nombre del servicio y debe escribirse en minúsculas.

Si este directorio existe, tiene prioridad y hace que se ignore `/etc/pam.conf`.

> **Correspondencia entre servicio y fichero**

El siguiente ejemplo muestra la idea básica del esquema por servicio: cada aplicación PAM usa el fichero cuyo nombre coincide con el nombre del servicio.

```
/etc/pam.d/login
/etc/pam.d/sshd
/etc/pam.d/su
/etc/pam.d/vsftpd
```

ESTRUCTURA DE LAS REGLAS PAM

FORMATO EN `/etc/pam.conf`

La documentación oficial define la siguiente estructura general para cada regla:

```
service type control module-path module-arguments
```

Las reglas suelen ocupar una sola línea, aunque pueden continuarse con una barra invertida al final de línea. Los comentarios empiezan con `#` y se extienden hasta el final de la línea.

FORMATO EN `/etc/pam.d/NOMBRE_SERVICIO`

En los ficheros dentro de `/etc/pam.d/` la sintaxis es la misma, salvo que desaparece el campo `service`, ya que el nombre del servicio viene dado por el propio nombre del fichero.

```
type control module-path module-arguments
```

Regla PAM en `/etc/pam.d`

El siguiente ejemplo muestra una línea típica en un fichero de servicio dentro de `/etc/pam.d`.

```
auth required pam_unix.so
```

SIGNIFICADO DE LOS CAMPOS

CAMPO `SERVICE`

En `/etc/pam.conf`, el campo `service` indica a qué servicio se aplica la regla. La documentación reserva el nombre especial `other` para definir reglas por defecto. Solo se asociarán a la aplicación actual las líneas que coincidan con su servicio o, si no existen, las entradas `other`.

CAMPO `TYPE`

El campo `type` indica a qué grupo de gestión pertenece la regla. La documentación oficial define cuatro grupos válidos:

Tipo	Función general	Idea práctica
`auth`	Autenticación e inicialización de credenciales.	Verifica identidad y puede preparar credenciales.
`account`	Gestión de cuenta no basada directamente en la contraseña.	Decide si la cuenta puede acceder en ese contexto.
`password`	Actualización del token de autenticación.	Se usa al cambiar contraseñas u otros secretos.
`session`	Operaciones de apertura y cierre de sesión.	Ejecuta acciones antes o después de prestar el servicio.

La propia documentación de `pam.conf(5)` y `pam.d(5)` describe cada grupo así:

- `auth`: se ocupa de comprobar que el usuario es quien dice ser y también puede conceder credenciales o privilegios asociados.
- `account`: se usa para restricciones de cuenta, por ejemplo horario, recursos disponibles o ubicación desde la que se accede.
- `password`: se usa para actualizar el token de autenticación asociado al usuario.
- `session`: se asocia a tareas que deben realizarse antes o después de que el usuario obtenga el servicio, como registrar eventos o montar directorios.

Además, la documentación indica que si el valor de `type` va precedido por un guion `-`, la biblioteca PAM no escribirá un mensaje en el registro del sistema si el módulo no puede cargarse porque no existe. Esto es útil con módulos opcionales que pueden no estar instalados en todos los equipos.

Tipo con guion inicial

El siguiente ejemplo muestra una regla en la que PAM no registrará como error la ausencia física del módulo si este no estuviera instalado.

```
-session optional pam_modulo_no_siempre_instalado.so
```

CAMPO `CONTROL`

El campo `control` es la pieza decisiva para entender cómo se evalúa una pila PAM. Indica qué debe hacer la biblioteca si el módulo devuelve un determinado resultado.

La documentación oficial distingue dos sintaxis:

- Una sintaxis **simple** basada en palabras clave históricas.
- Una sintaxis **extendida** basada en pares `valor=acción` entre corchetes.

SINTAXIS SIMPLE

Los valores simples documentados son `required`, `requisite`, `sufficient`, `optional`, `include` y `substack`.

Control	Comportamiento documentado
`required`	Si falla, el resultado final será fallo, pero PAM seguirá evaluando el resto de módulos del mismo tipo.
`requisite`	Si falla, PAM termina inmediatamente la evaluación de esa pila y devuelve fallo.
`sufficient`	Si tiene éxito y no ha fallado antes ningún `required`, PAM devuelve éxito inmediatamente y no sigue evaluando más módulos de esa pila. Si falla, el fallo se ignora.
`optional`	Su éxito o fallo solo importa si es el único módulo de esa pila para ese servicio y tipo.
`include`	Incluye todas las líneas del tipo indicado desde otro fichero de configuración.
`substack`	Incluye un sub-bloque de otro fichero, pero con reglas de control propias respecto a `done`, `die`, saltos y `reset`.

Estas cuatro palabras clave clásicas equivalen, según la documentación oficial, a expresiones concretas de la sintaxis extendida:

Forma simple	Equivalencia oficial
`required`	`[success=ok new_authtok_reqd=ok ignore=ignore default=bad]`
`requisite`	`[success=ok new_authtok_reqd=ok ignore=ignore default=die]`
`sufficient`	`[success=done new_authtok_reqd=done default=ignore]`
`optional`	`[success=ok new_authtok_reqd=ok default=ignore]`

SINTAXIS EXTENDIDA

La sintaxis extendida tiene esta forma:

```
[value1=action1 value2=action2 ...]
```

Cada `value` representa un código de retorno del módulo, por ejemplo `success`, `auth_err`, `user_unknown`, `ignore`, `try_again` o `default`. Cada `action` indica cómo debe reaccionar PAM ante ese resultado.

Las acciones documentadas oficialmente son las siguientes:

Acción	Significado
`ignore`	El resultado de ese módulo no contribuye al resultado final.
`bad`	El resultado cuenta como fallo. Si es el primer fallo relevante, su estado se usará para el resultado global.
`die`	Igual que `bad`, pero además termina inmediatamente la evaluación.
`ok`	El resultado contribuye positivamente al estado global si no existía ya un fallo relevante previo.
`done`	Igual que `ok`, pero además termina inmediatamente la evaluación salvo que antes haya existido un fallo no ignorado.

Acción	Significado
`N`	Salta los siguientes `N` módulos de la pila.
`reset`	Borra el estado acumulado de la pila y continúa desde el siguiente módulo.

Si un código de retorno no aparece de forma explícita y tampoco existe `default`, su acción por defecto es `bad`.

CAMPO `MODULE-PATH`

Es la ruta del módulo que debe cargarse. Puede ser una ruta absoluta o una ruta relativa respecto a la ubicación por defecto de los módulos PAM, que la documentación sitúa en `/lib/security/` o `/lib64/security/`, según la arquitectura.

CAMPO `MODULE-ARGUMENTS`

Son argumentos adicionales que modifican el comportamiento del módulo. La sintaxis exacta depende de cada módulo concreto y se documenta en su propia página de manual.

La documentación oficial añade que, si un argumento necesita contener espacios, debe escribirse entre corchetes. Además, dentro de ese mecanismo puede escaparse `]` como `\]`.

CÓMO SE EVALÚA UNA PILA PAM

IDEA GENERAL DE PILA O STACK

Una de las características más importantes de PAM es el apilamiento de reglas. Para un mismo servicio y para un mismo tipo (`auth`, `account`, `password` o `session`), pueden definirse varias líneas consecutivas. PAM las evalúa como una pila y combina sus resultados conforme al campo `control`.

Pila simple de autenticación

El siguiente ejemplo no pretende recomendar módulos concretos, sino mostrar que varias reglas del mismo tipo se evalúan como una pila con una semántica acumulativa.

```
auth required   pam_modulo_a.so
auth sufficient pam_modulo_b.so
auth required   pam_modulo_c.so
```

En una pila así, no basta con leer los módulos: hay que leer la pila completa junto con sus controles. Un `required` fallido puede quedar "pendiente" hasta el final; un `requisite` corta la evaluación; un `sufficient` puede dar éxito inmediato si no existía un fallo previo relevante.

EVALUACIÓN DE `REQUIRED`

Cuando una regla `required` falla, la pila sigue ejecutándose, pero el resultado final ya quedará comprometido. El objetivo es permitir que se ejecuten los módulos restantes aunque la autenticación vaya a terminar fallando, por ejemplo para no revelar demasiado pronto en qué punto exacto se produjo el fallo.

Fallo diferido hasta el final

En el siguiente esquema, aunque falle la primera línea, PAM seguirá evaluando las demás reglas del mismo tipo antes de devolver el fallo final.

```
auth required pam_modulo_1.so
auth optional pam_modulo_2.so
auth required pam_modulo_3.so
```

EVALUACIÓN DE `REQUISITE`

`requisite` se comporta como `required`, pero con corte inmediato en caso de fallo. La propia documentación subraya que esta semántica puede ser útil cuando no interesa seguir adelante si ya se ha producido un error determinante.

EVALUACIÓN DE `SUFFICIENT`

Si un módulo `sufficient` tiene éxito y antes no ha fallado ningún `required`, PAM puede terminar inmediatamente con éxito. Si ese módulo falla, el fallo se ignora y la pila continúa.

La consecuencia práctica es que `sufficient` no significa "obligatorio y bastante", sino "bastante si funciona y no existe ya un fallo previo relevante".

EVALUACIÓN DE `OPTIONAL`

Una línea `optional` apenas influye en el resultado global salvo que sea la única línea de la pila para ese servicio y ese tipo. Se utiliza sobre todo para acciones accesorias cuya ejecución no debe decidir por sí misma el éxito o fracaso de todo el proceso.

DIFERENCIA ENTRE `INCLUDE` Y `SUBSTACK`

Ambos mecanismos permiten reutilizar reglas de otro fichero, pero la documentación oficial insiste en que no son equivalentes.

- `include` inserta todas las líneas del tipo indicado desde otro fichero y las integra plenamente en la pila principal.
- `substack` también incorpora reglas de otro fichero, pero las acciones `done` y `die` solo afectan al sub-bloque, no al resto de la pila completa.
- Además, los saltos numéricos dentro de un `substack` no pueden hacer que la evaluación salga fuera de él, y un `reset` restablece el estado al inicio de la evaluación del substack.

Inclusión de reglas comunes

El siguiente ejemplo muestra la forma típica de reutilizar reglas comunes desde otro fichero.

```
auth include common-auth
account include common-account
```

```
session include common-session
```

ORDEN DE EVALUACIÓN PRÁCTICO

Para leer correctamente un fichero PAM conviene seguir siempre este orden mental:

- Identificar el **servicio** al que pertenece el fichero.
- Separar las reglas por **tipo**: `auth`, `account`, `password` y `session`.
- Leer cada pila de arriba abajo.
- Interpretar cada línea junto con su **control**, no como una regla aislada.
- Tener en cuenta inclusiones (`include`) y subpilas (`substack`) antes de sacar conclusiones.
- Recordar que el resultado final depende del estado acumulado de la pila y no solo del último módulo ejecutado.

ERRORES DE SINTAXIS Y PRECAUCIONES

La documentación oficial advierte de que cualquier línea mal formada en los ficheros de configuración tiende, por precaución, a hacer fallar el proceso de autenticación. Además, se registra un mensaje de error mediante `syslog`.

Esto hace que la edición de ficheros PAM deba abordarse con especial cuidado: un error sintáctico no suele dejar un comportamiento degradado sino, con frecuencia, un servicio directamente inutilizable.

QUÉ NO HACE ESTE ANEXO

Este anexo explica la arquitectura y la lógica de evaluación de PAM, pero no entra todavía a describir módulos concretos. Ese sería el siguiente nivel: estudiar, por separado, qué hace cada módulo y en qué tipo de pila debe usarse.

REFERENCIAS

- Manual oficial `pam.conf(5)` y `pam.d(5)` de Linux-PAM en Debian: https://manpages.debian.org/bookworm/libpam-runtime/pam.conf.5.en.html
- Manual oficial `pam.d(5)` de Linux-PAM en Debian: https://manpages.debian.org/bookworm/libpam-runtime/pam.d.5.en.html
- Documentación oficial del proyecto Linux-PAM: https://www.linux-pam.org/